DIGITAL SYSTEMS

DESIGN AND VERIFICATION

A WHAT?

- PLD
- CPLD
- FPGA
- GAL
- FPLA
- PAL
- ASIC

A WHAT?

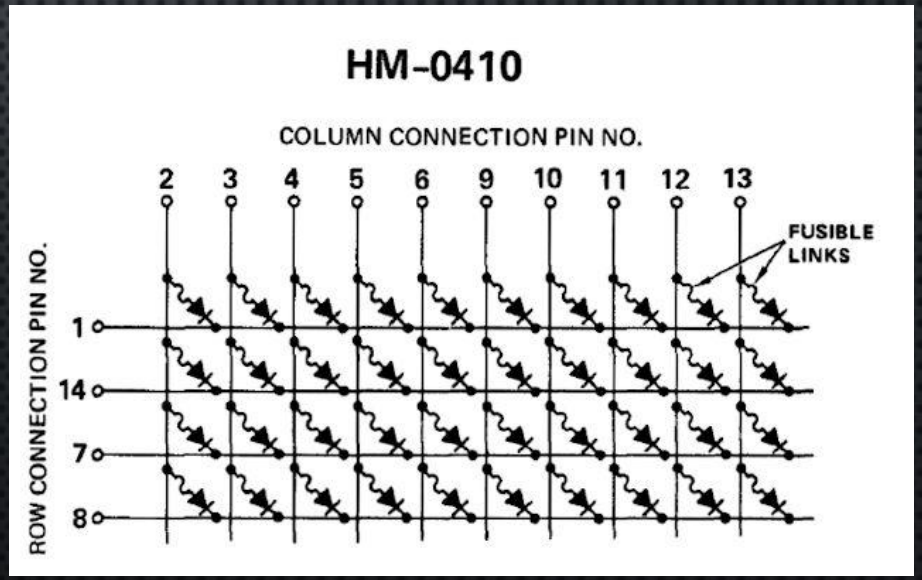
- Programmable Logic Device
- Complex - PLD
- Field Programmable Gate Array
- Gate Array Logic
- Field Programmable Logic Array
- Programmable Array Logic
- Application Specific IC

A LONG LONG TIME AGO…

50s-60s-70s

DRL

Stanley Frankel

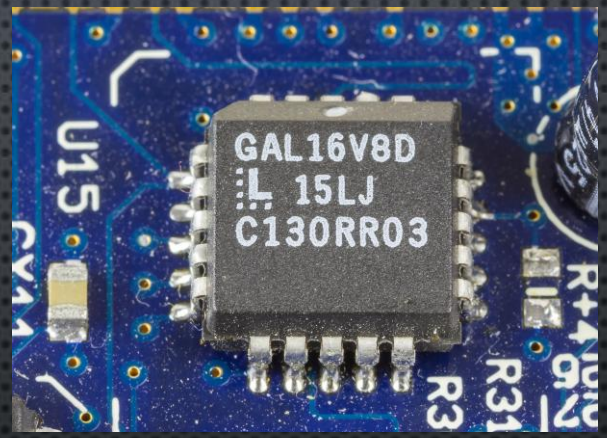


HISTORY
EVOLUTION

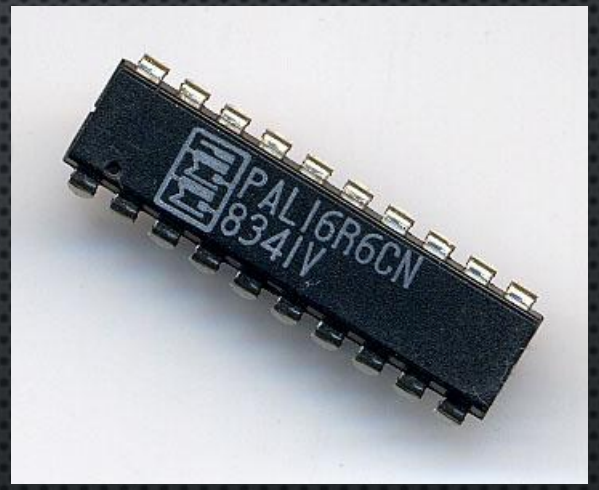
80s

Fixed Arrays

Programmable Gates



1983: GAL



1978: PAL

THE FAMOUS…

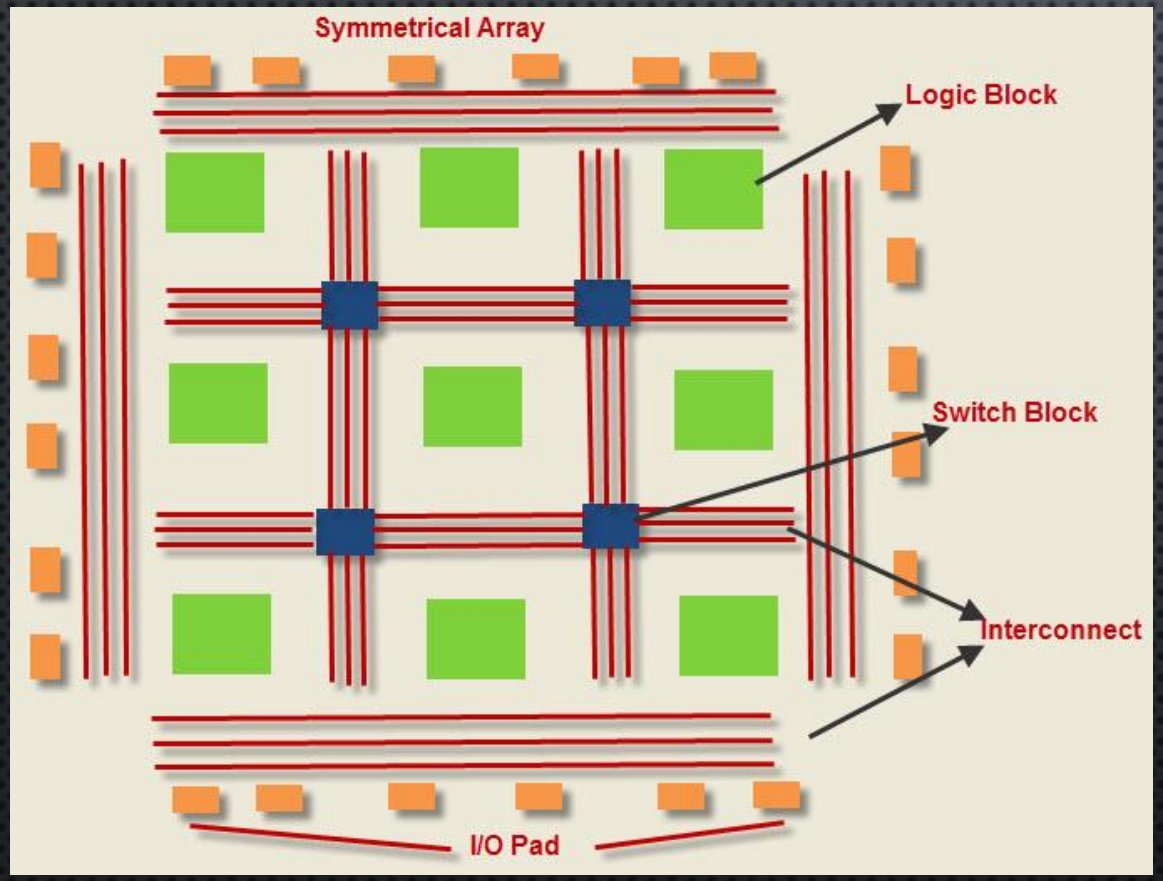
FPGAs

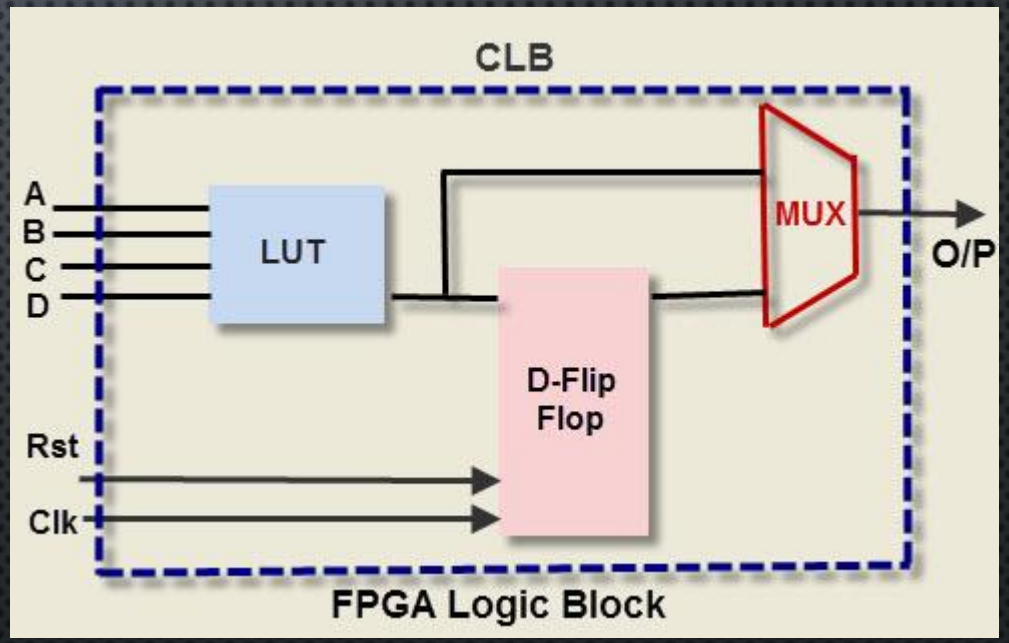
Programmable Rootable Matrix

LUTs

1985: FPGA

ANATOMY OF AN FPGA





WHY WOULD WE CHOSE AN FPGA?

+DETERMINISTIC TIMING

+ULTRA FAST

+ DESIGN EVERYTHING

+ COMPLEX

+ HIGH ENERGY CONSUMPTION

+ NON INTUITIVE CODE

+EXPENSIVE

De10Lite

7,000 MXN

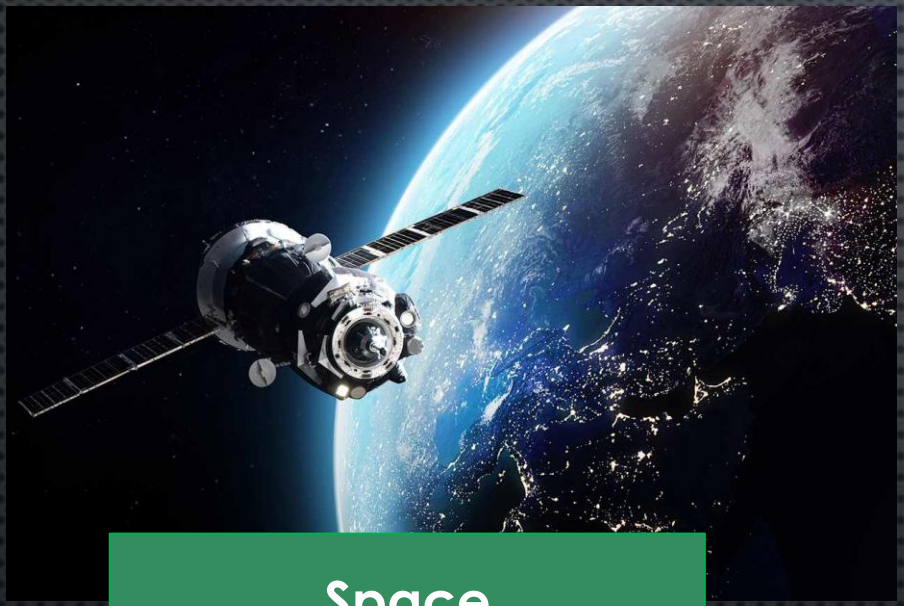
PARALLEL

Nucleo STM32

500 MXN

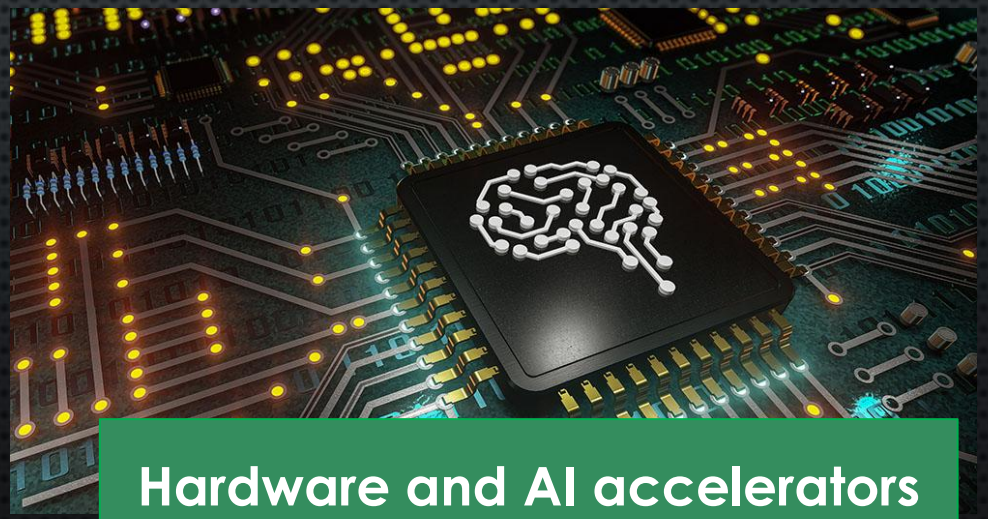
LINE X LINE

REAL LIFE APPLICATIONS



Space

Telecomm



Hardware and AI accelerators

¿¿¿¿ HDL ?????
VERILOG VS VHDL

+ Hardware Description Language

  + C/C++ // Verilog/VHDL

+ Verilog -> More focused in hardware modeling

+ VHDL -> VHDL more verbose and different sintax

+ Verilog is often more used in industry environment

VHDL:

```
process ({S0,S1},A,B,C,D)
begin
   case {S0,S1}, is
      when "00" => Y <= A;
      when "01" => Y <= B;
      when "10" => Y <= C;
      when "11" => Y <= D;
      when others => Y <= A;
   end case;
end process;
```

Verilog:

```

always @({S0,S1}, A, B, C, D)
      case ({S0,S1})
         2'b00: Y = A;
         2'b01: Y = B;
         2'b10: Y = C;
         2'b11: Y = D;
      endcase

```

DE10LITE INTRO

Device: Intel MAX 10 (10M50DAF484C7G).

Logic Elements (LEs): 50,000.

Internal Memory: 1,638 Kbit of M9K blocks.

Internal Flash: Dual-boot capability (remembers code even when unplugged).

System Clock: 50 MHz crystal oscillator

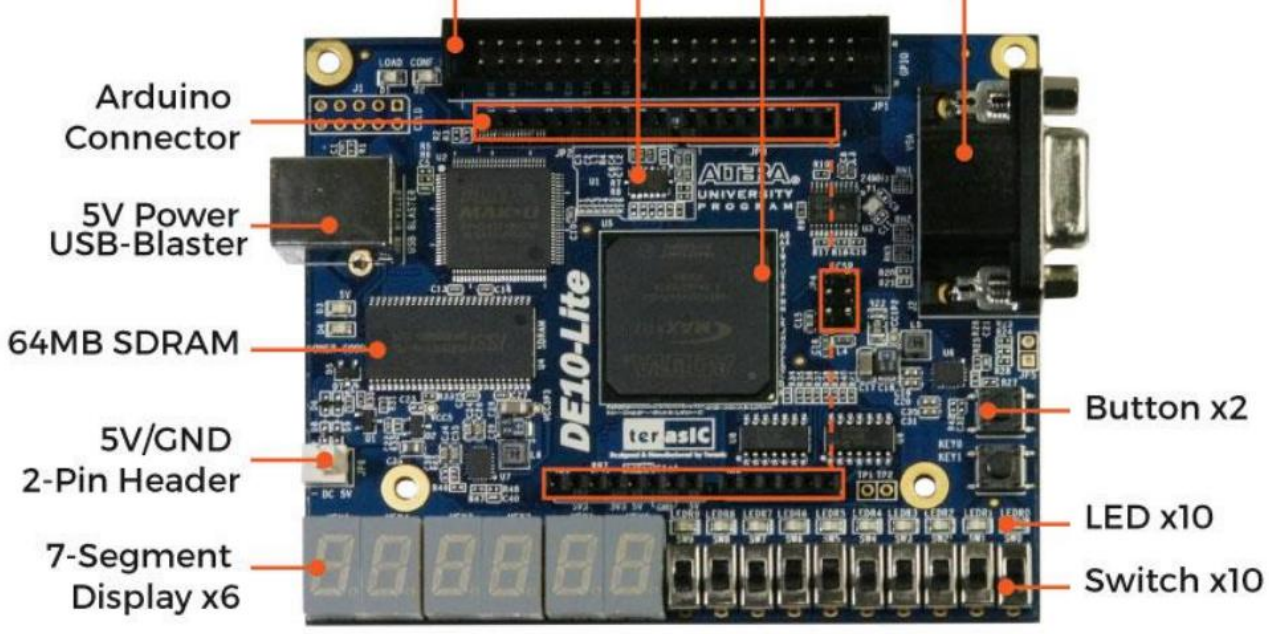
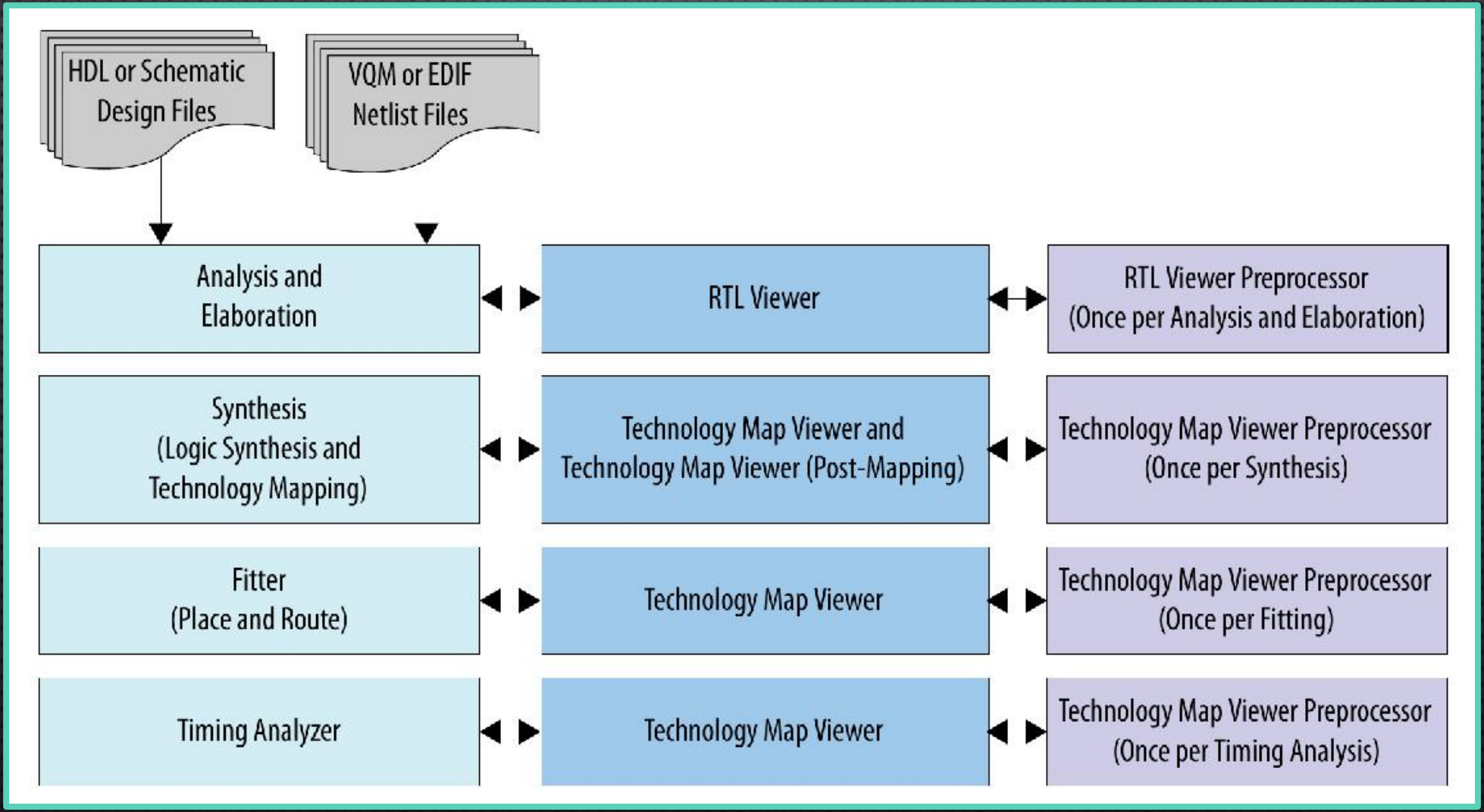
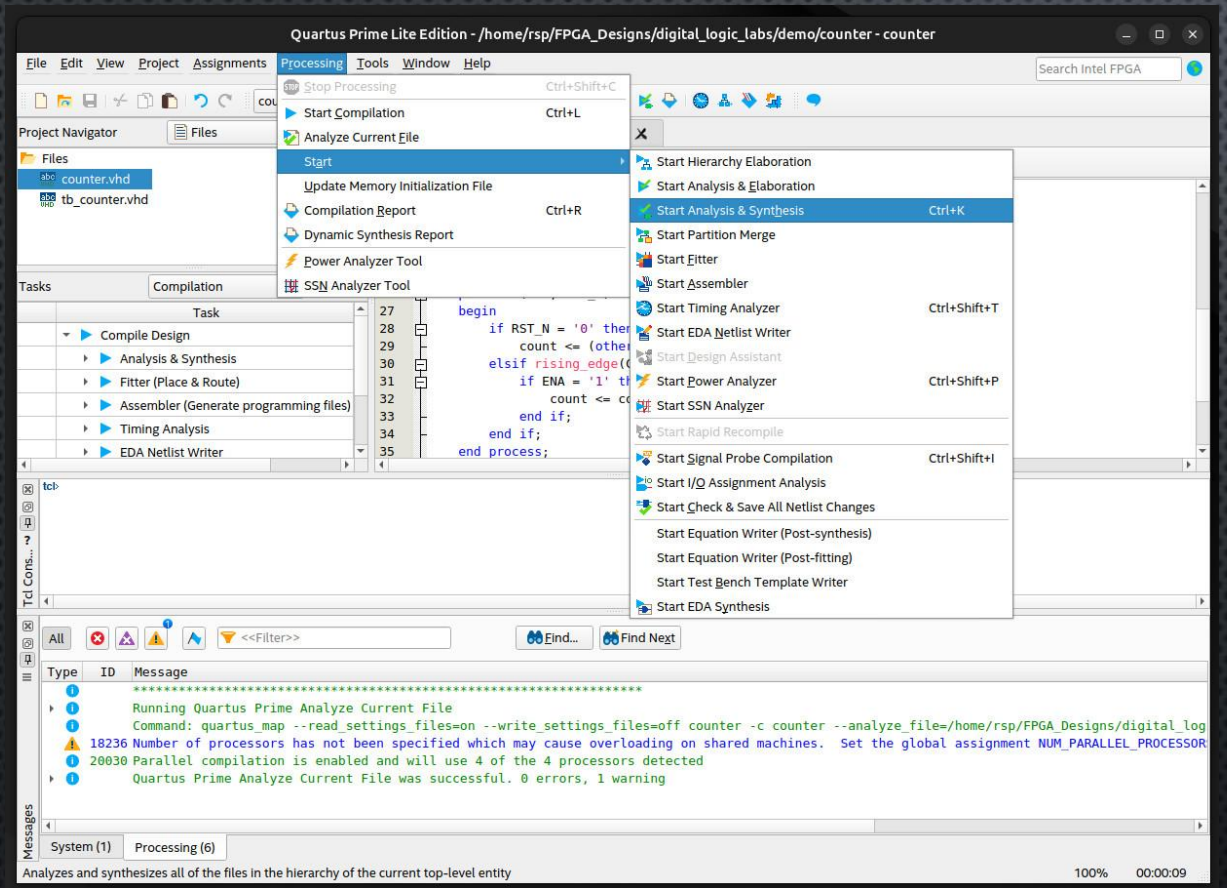


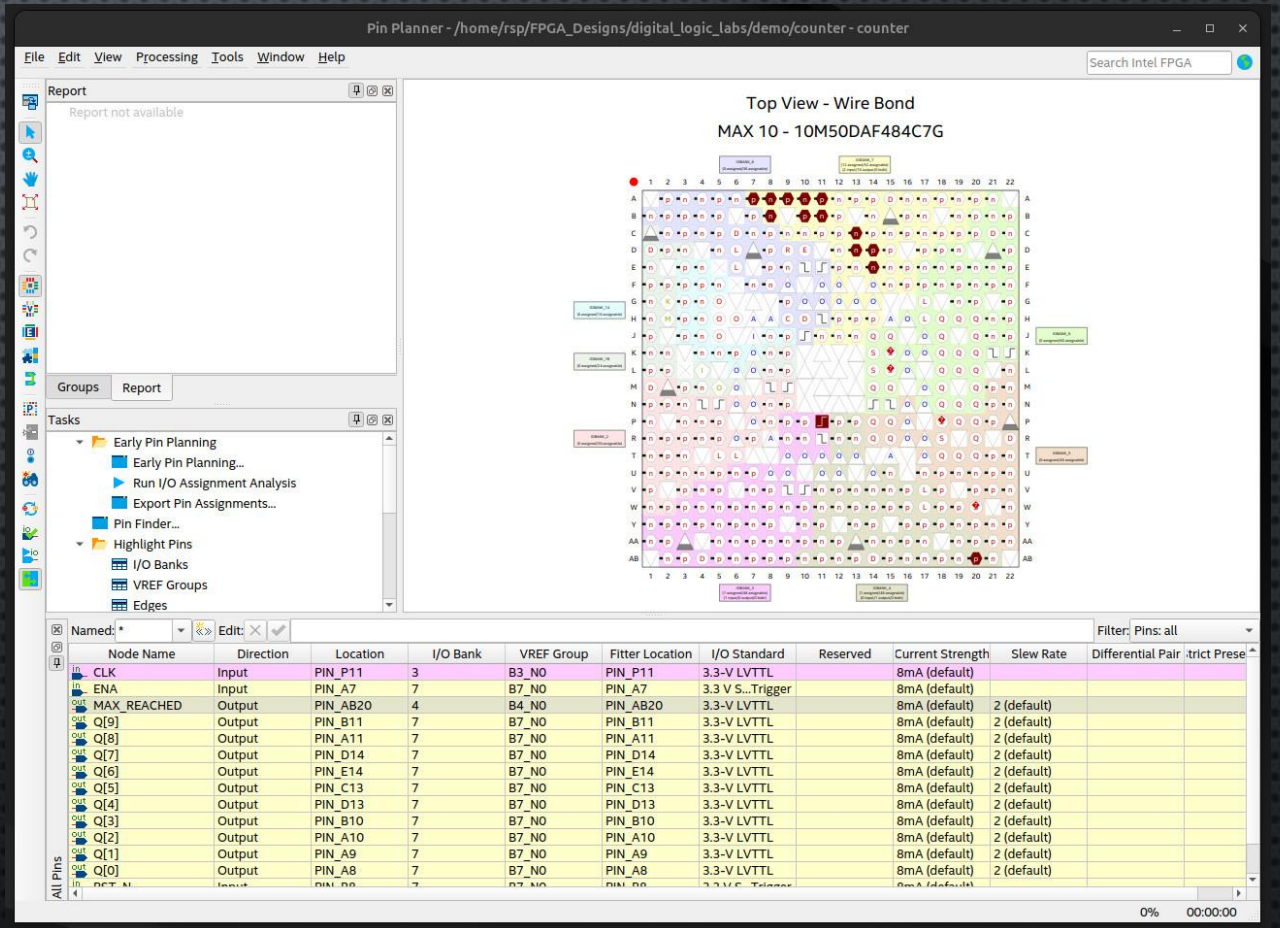
Figure 1-2 Development Board (top view)

INTEL QUARTUS PRIME - WORKFLOW



INTEL QUARTUS PRIME - WORKFLOW





VERILOG FUNDAMENTALS - DATA TYPES

A bit can take on one of four values

Value	Meaning
0	Logic Zero
1	Logic One
X	Unknown logic value
Z	High Impedance, Floating

An X bit might be a 0, 1, Z, or in transition.

We can set bits to be X in situations where we don't care what the value is.

This can help catch bugs and improve synthesis quality

BIT LITERALS

4'b10_11

Base Format

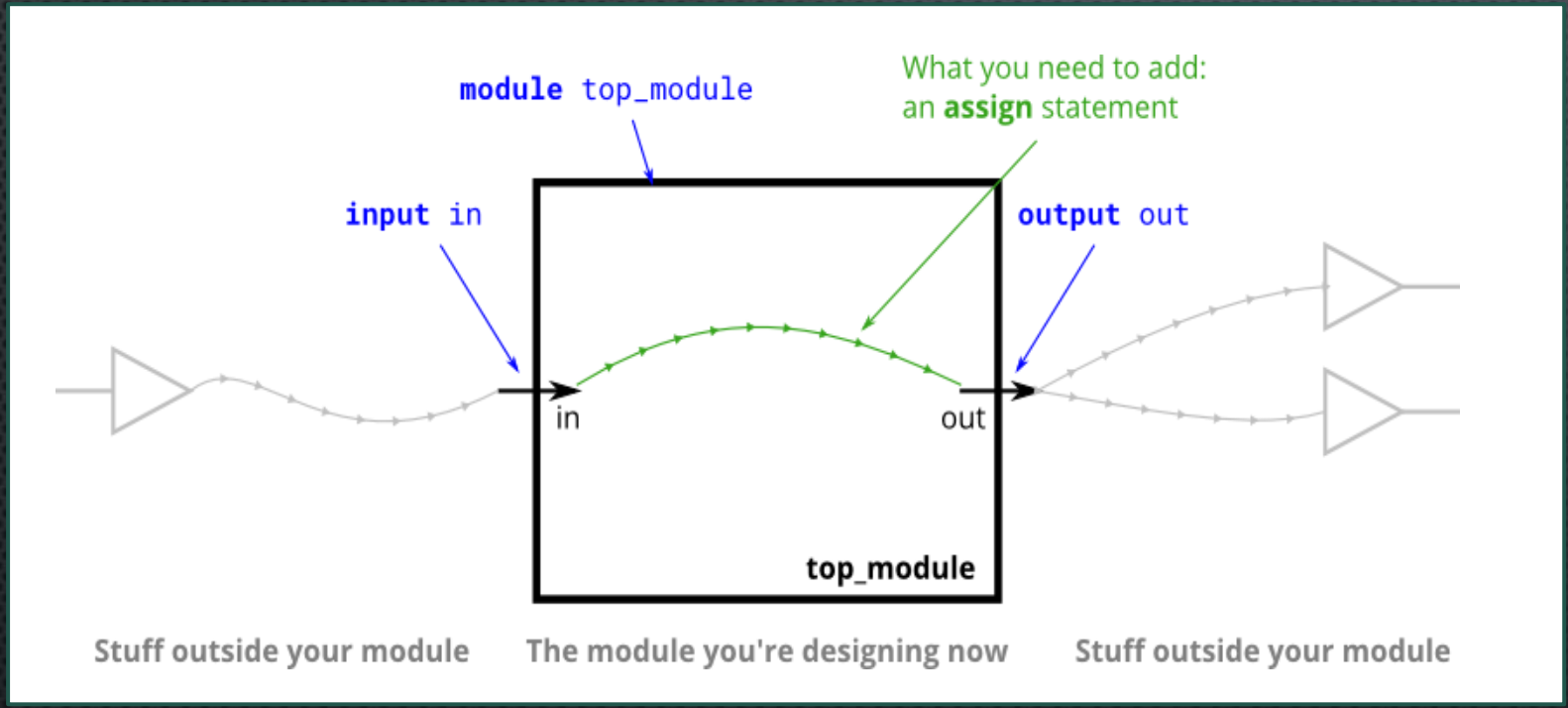
Size in bits

Binary Literals
- 2'b01

Hexadecimal Literals
- 16'haxxx

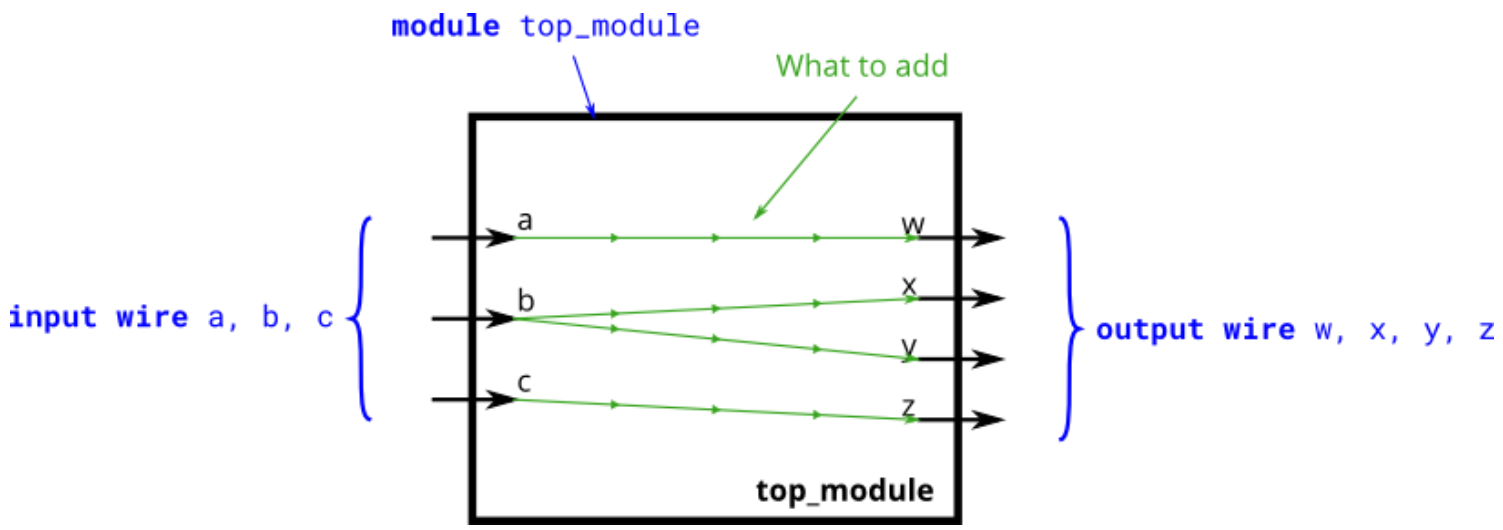
Decimal Literals
- 32'd42

VERILOG FUNDAMENTALS – WIRES



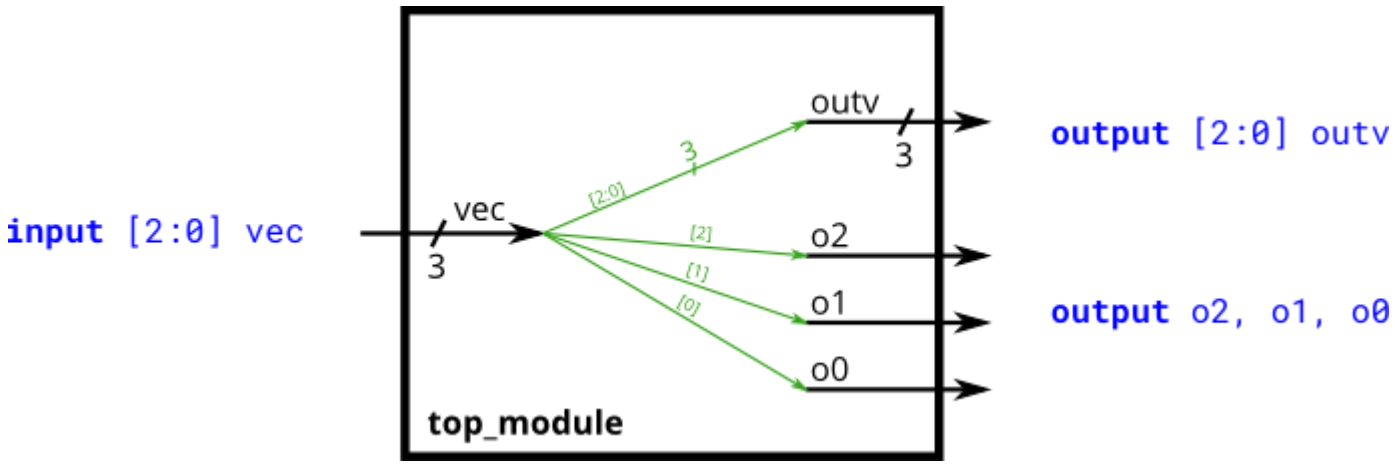

```
module top_module
  (
  input in,
  output out
   );

  assign out = in;

endmodule
```

VERILOG FUNDAMENTALS – WIRES




```
module top_module
  (
  input
  output
   );

   ;

endmodule
```

VERILOG FUNDAMENTALS – VECTORS




```
module top_module (
    input wire [2:0] vec,
    output wire [2:0] outv,
    output wire o2,
    output wire o1,
    output wire o0
    );

  assign outv = vec;
  assign o0 = vec[0];
  assign o1 = vec[1];
  assign o2 = vec[2];

endmodule
```

VERILOG FUNDAMENTALS – VECTORS +

Vectors must be declared:

type [upper:lower] vector_name;

type -> datatype of the vector.
If you are declaring a input or output port, the type can additionally include the port type as well.

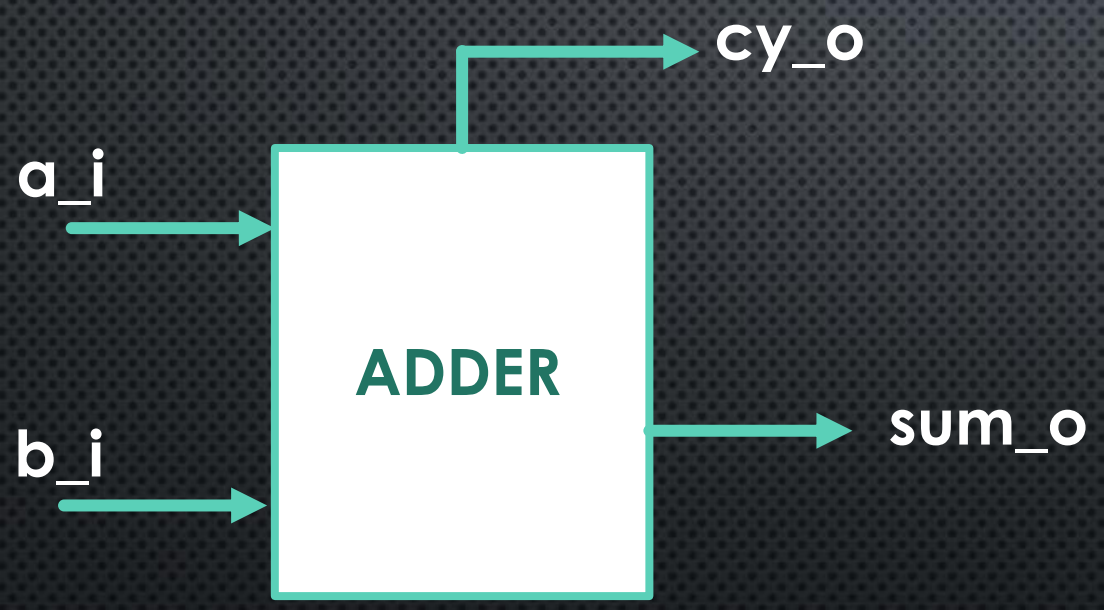
wire [7:0] w; // 8-bit wire
reg [4:1] x; // 4-bit reg
output reg [0:0] y; // 1-bit reg that is also an output port (this is still a vector)

input wire [3:-2] z; // 6-bit wire input (negative ranges are allowed)

output [3:0] a; // 4-bit output wire. Type is 'wire' unless specified otherwise.

wire [0:7] b; // 8-bit wire where b[0] is the most-significant bit.

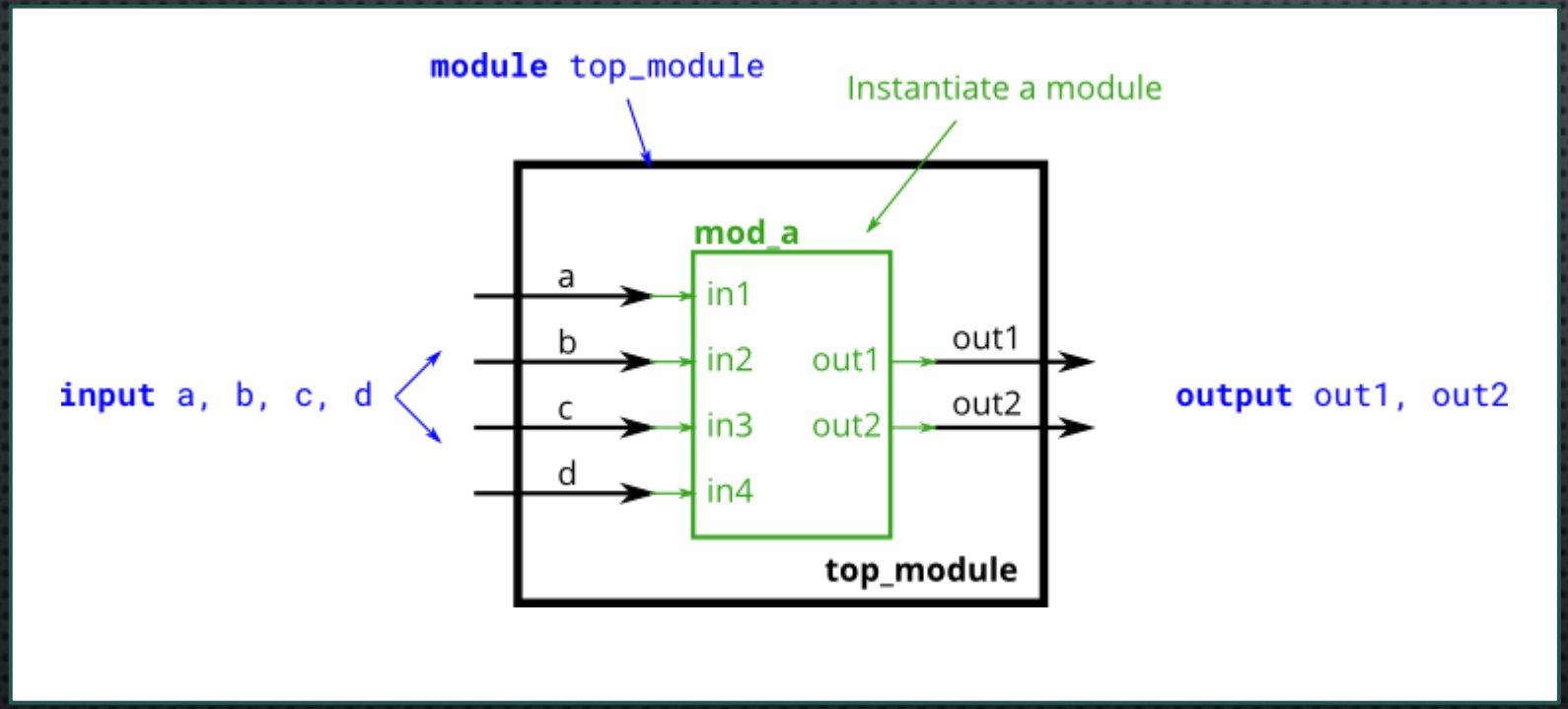
VERILOG FUNDAMENTALS – MODULES




```
module adder (
        input [3:0] a_i,
        input [3:0] b_i,
        output cy_o,
        output [3:0] sum_o );

// HDL modeling of
// adder functionality

endmodule
```

Ports must have a direction and a bitwidth

VERILOG FUNDAMENTALS – CONNECTING MODULES

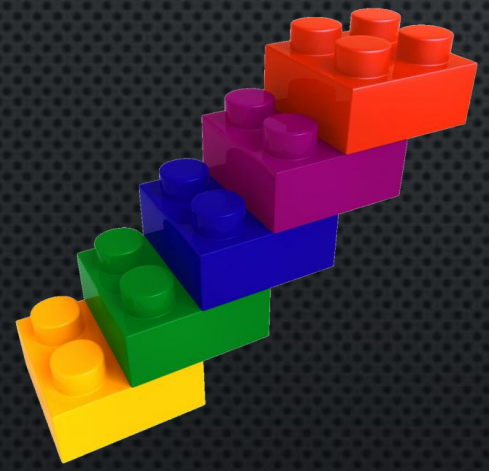



```
module top_module (
    input a,
    input b,
    input c,
    input d,
    output out1,
    output out2
    );

 mod_a inst(
    .out1(out1),
    .out2(out2),
    .in1(a),
    .in2(b),
    .in3(c),
    .in4(d)
 );

endmodule
```

VERILOG FUNDAMENTALS –DESIGN TYPES

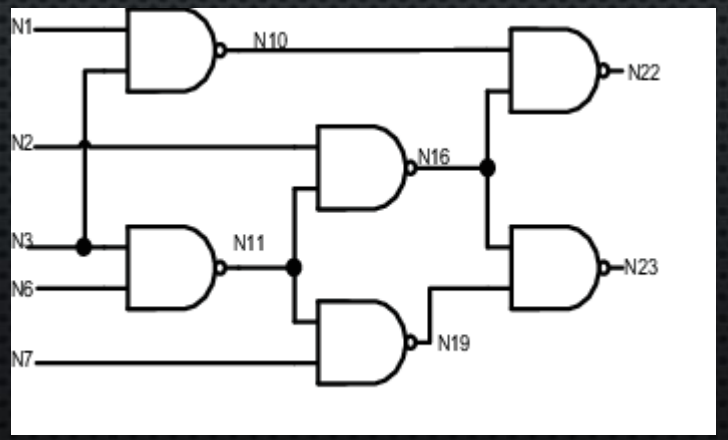
Structural

Manually instantiate modules and connect them using wire



Dataflow

Simple Combinational Logic: **assign** keyword and **Boolean operators** (&, |, ^, ~)



Behavioral

Always blocks and **if/else** or **case** statements.
(Complex Logic)

EX1 – DIGITAL VAULT!

```
module digitalvault (
    input  wire [9:0] SW,    // 10 Switches
    output wire [9:0] LEDR   // 10 LEDs
);
    // Wire Declaration
    wire master;
    wire password;
    wire vault_unlocked;

    // Assign logic to our Wires
    assign master   = SW[9];                  // Master Switch
    assign password = (SW[3:0] == 4'b1011);   // Secret Key (1011)

    // Final Condition
    assign vault_unlocked = master & password;

    // Mapping the results to the physical LEDs
    assign LEDR[9]   = vault_unlocked;  // Lock
    assign LEDR[8]   = master;          // Master Status
    assign LEDR[3:0] = SW[3:0];         // Input Feedback

    // Turn off unused LEDs to avoid Driver Warnings
    assign LEDR[7:4] = 4'b0000;

endmodule
```

HOMEWORK (OPTIONAL BTW)
HDL BITS

Basics

- Simple wire ✓
- Four wires ✓
- Inverter
- AND gate
- NOR gate
- XNOR gate
- Declaring wires
- 7458 chip

Vectors

- Vectors ✓
- Vectors in more detail ✓
- Vector part select
- Bitwise operators
- Four-input gates
- Vector concatenation operator
- Vector reversal 1
- Replication operator
- More replication

Modules: Hierarchy

- Modules
- Connecting ports by position
- Connecting ports by name ✓
- Three modules
- Modules and vectors
- Adder 1
- Adder 2
- Carry-select adder
- Adder-subtractor